МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ

по курсу
«Разработка веб-приложений»

ТЕМА

«Разработка веб-приложения для учёта расходов и доходов семейного бюджета»

Выполнил: Момина Антонина Алексеевна
Группа: 241-327

Проверил: Кружалов Алексей Сергеевич

Москва, 2026

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

УТВЕРЖДАЮ
заведующая кафедрой
«Инфокогнитивные технологии»

_____ / Е. А. Пухова /
«___» _____ 2026 г.

ЗАДАНИЕ

на выполнение курсовой работы (проекта)

Моминой Антонине Алексеевне,

обучающемуся группы 241-327,

направления подготовки 09.03.01 «Информатика и вычислительная техника»

по дисциплине «Разработка веб-приложений»

на тему «**Разработка веб-приложения для учёта расходов и доходов семейного бюджета**»

1. Исходные данные к работе (проекту): информационные ресурсы в сети интернет, научные публикации в открытой печати.

2. Содержание задания по курсовой работе (проекту) – перечень вопросов, подлежащих разработке:

Разрабатываемый вопрос	**Объем, %**	**Срок**	**Примечание**
Раздел 1. Анализ предметной области			
Задача 1.1. Обзор существующих программных продуктов по теме работы	10	22.03.2026	
Задача 1.2. Анализ программных инструментов разработки веб-приложений	7	26.03.2026	
Задача 1.3. Формулировка цели и задач работы	8	30.03.2026	
Раздел 2. Проектирование веб-приложения			
Задача 2.1. Анализ целевой аудитории	5	02.04.2026	
Задача 2.2. Описание функциональности приложения (диаграмма вариантов использования, user story)	7	06.04.2026	

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Задача 2.3. Проектирование модели данных (ER-диаграмма, логическая и физическая схемы БД)	8	10.04.2026	
Задача 2.4. Разработка макетов страниц (Wireframe)	5	14.04.2026	
Раздел 3. Разработка веб-приложения			
Задача 3.1. Разработка базовой структуры приложения и вёрстка шаблонов страниц	8	21.04.2026	
Задача 3.2. Реализация системы аутентификации и авторизации пользователей с ролями	6	27.04.2026	
Задача 3.3. Реализация CRUD-интерфейса для основных сущностей (Transaction, Category, Budget, Receipt)	7	04.05.2026	
Задача 3.4. Реализация обработки и анализа данных и загрузки/выгрузки файлов (чеки, отчёты)	12	12.05.2026	
Задача 3.5. Разработка системы статистики и визуализации (дашборды, диаграммы)	10	18.05.2026	
Раздел 4. Оформление итогов работы			
Задача 4.1. Создание Git-репозитория с кодом проекта	3	22.03.2026	
Задача 4.2. Деплой приложения на хостинг	4	26.05.2026	
Задача 4.3. Оформление отчёта о проделанной работе	3	26.05.2026	

Руководитель курсовой работы (проекта):
старший преподаватель кафедры «Инфокогнитивные технологии»

«____» _____ 2026 г. _____ А. С. Кружалов
(подпись)

Дата выдачи задания «1» апреля 2026 г.

Дата сдачи выполненной работы «____» _____ 2026 г.

Задание принял к исполнению

«1» апреля 2026 г. _____ А. А. Момина
(подпись)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **5**

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ **8**

1.1 Обзор существующих программных продуктов 8

1.2 Анализ программных инструментов разработки 9

1.3 Формулировка цели и задач работы 10

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ **12**

2.1 Анализ целевой аудитории 12

2.2 Описание функциональности приложения 12

2.3 Проектирование модели данных 12

2.4 Разработка макетов страниц 14

ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ **15**

3.1 Разработка базовой структуры приложения и вёрстка 15

3.2 Реализация модуля авторизации 15

3.3 Интеграция картографических сервисов 15

ЗАКЛЮЧЕНИЕ **17**

ВВЕДЕНИЕ

В современных условиях эффективное управление личными и семейными финансами является важной задачей для большинства людей. Однако многие семьи сталкиваются с трудностями при контроле доходов и расходов.

Проблема. Основная проблема состоит в том, что у людей часто нет удобного и надёжного инструмента для систематического учёта финансов. Многие ведут записи в заметках или простых таблицах Excel. Это приводит к неконтролируемым тратам, невозможности правильно спланировать бюджет и снижению финансовой стабильности. Кроме того, существующие приложения часто неудобны для совместного использования несколькими людьми, имеют ограниченные возможности аналитики и не всегда позволяют работать с чеками и документами.

Актуальность. Тема учёта семейных финансов остаётся актуальной в 2026 году. Согласно исследованию Аналитического центра НАФИ и страховой компании «Росгосстрах Жизнь» (2025) [1], индекс финансовой грамотности россиян снизился до 12,61 балла из 21 возможного. Доля граждан с низким уровнем финансовой грамотности выросла до 34 %, а среди молодёжи 18–34 лет составляет 51 %. Почти половина работающих россиян регулярно испытывает финансовые трудности.

В условиях прогнозируемой инфляции 4,5–5,5 % (по данным Банка России на 2026 год) [2] семьи всё больше нуждаются в удобном инструменте для ведения общего семейного бюджета, отслеживания расходов всех членов семьи, загрузки чеков и получения наглядной статистики.

Степень разработанности проблемы. На сегодняшний день существует множество приложений для учёта финансов. Они позволяют вести учёт транзакций, ставить бюджеты и смотреть простые отчёты. Однако большинство из них имеют существенные недостатки: платные премиум-функции, слабая поддержка совместной работы нескольких пользователей, ограниченные возможности загрузки файлов и не всегда удобная аналитика. Готовых открытых веб-решений, которые одновременно включают систему ролей пользователей, полноценный CRUD-интерфейс, загрузку и выгрузку файлов чеков и глубокую агрегацию данных, относительно немного. Это определяет место и актуальность настоящей курсовой работы.

Объект исследования — процесс учёта и анализа доходов и расходов

семейного бюджета с помощью веб-технологий.

Предмет исследования — методы и средства разработки веб-приложения для управления личными финансами, включающего аутентификацию, авторизацию по ролям, CRUD-операции, агрегацию данных и работу с файлами.

Цель работы — разработать веб-приложение для учёта расходов и доходов семейного бюджета, которое обеспечит удобный совместный контроль финансов, аналитику и безопасность данных.

Для достижения поставленной цели необходимо решить следующие **задачи**:

- проанализировать существующие программные продукты и технологии разработки веб-приложений для управления финансами;
- определить требования целевой аудитории и спроектировать функциональность приложения (диаграммы вариантов использования, User Story);
- разработать модель данных (ER-диаграмму, логическую и физическую схемы базы данных);
- спроектировать пользовательский интерфейс (wireframes);
- реализовать систему аутентификации и авторизации с проверкой прав доступа по ролям;
- разработать CRUD-интерфейс для основных сущностей (транзакции, категории, бюджеты);
- реализовать механизмы загрузки и выгрузки файлов (чеки, отчёты) и агрегацию данных с визуализацией статистики;
- провести тестирование, деплой приложения и оформить отчётную документацию.

Актуальность темы. В работе применялись методы системного анализа, сравнительный анализ существующих решений, структурное и объектно-ориентированное проектирование. Для реализации использовались современные веб-технологии (backend, frontend и база данных). Визуализация данных

выполнялась с помощью специализированных библиотек. Тестирование проводилось по функциональным и нефункциональным требованиям.

Структура отчёта соответствует логике поставленных задач и включает анализ предметной области, проектирование, описание реализации и заключение.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор существующих программных продуктов

На рынке представлено большое количество приложений для учёта личных и семейных расходов. Большинство из них — это мобильные приложения (Android/iOS). Полноценных веб-приложений, специально ориентированных на совместный семейный бюджет, на 2026 год очень мало.

Для сравнения были выбраны наиболее популярные решения: Дзен-мани, CoinKeeper, Monefy и Money Lover. В таблице ниже приведены их ключевые характеристики с точки зрения семейного использования.

Таблица 1.1 – Сравнение популярных приложений для учёта финансов

Характеристика	**Дзен-мани**	**Monefy**	**Money Lover**
Платформа	Мобильное	Мобильное	Мобильное + Web
Совместный бюджет	Полноценный	Сильно ограничен	Есть (в Premium)
Загрузка чеков	Есть (QR-сканирование)	Ограничено	Есть (фото)
Веб-версия	Ограниченная	Отсутствует	Полноценная
Стоимость ключевых функций	Премиум-подписка (многие отчёты и прогнозы)	Премиум для расширенной аналитики и синхронизации	Unlimited wallets и shared budgets — только Premium

Как видно из таблицы, у каждого рассмотренного приложения есть существенные недостатки именно при семейном использовании:

- **Дзен-мани** — лучший функционал совместного бюджета, но большинство важных отчётов, прогнозов и расширенной аналитики доступны только по платной подписке.
- **Monefy** — очень простой и интуитивный интерфейс, но совместный доступ сильно ограничен, а полноценной веб-версии нет.
- **Money Lover** — имеет веб-версию, однако создание нескольких общих кошельков и расширенные возможности бюджетирования также требуют платной подписки.

Таким образом, несмотря на обилие мобильных решений, на рынке практически отсутствует удобное, бесплатное (или полностью функциональное в базовой версии) веб-приложение, которое одновременно предоставляет:

- полноценную совместную работу нескольких пользователей,
- систему ролей и разграничения прав доступа,
- удобную загрузку и выгрузку файлов (чеков, отчётов),
- глубокую агрегацию данных и наглядную статистику,
- доступ с любого устройства без установки приложений.

Вывод. Разработка собственного веб-приложения для учёта семейных расходов позволит устранить указанные недостатки существующих решений и создать удобный, бесплатный в базовой функциональности инструмент, специально ориентированный на совместное использование всей семьёй.

1.2 Анализ программных инструментов разработки

Для разработки веб-приложения были рассмотрены следующие технологии и стеки:

Backend:

- Python 3.12 + Flask 3.1 — лёгкий и гибкий микрофреймворк, который позволяет быстро создавать RESTful-приложения и CRUD-интерфейсы.
- Flask-SQLAlchemy — для удобной работы с базой данных через ORM.

Frontend:

- HTML5 + CSS3 + JavaScript с Bootstrap 5.3 — современный CSS-фреймворк, обеспечивающий быструю адаптивную и responsive вёрстку без излишней сложности.
- Chart.js — библиотека для создания интерактивных диаграмм и графиков финансовой статистики.

База данных:

- SQLite — встроенная файловая СУБД, идеально подходящая для прототипирования, разработки и демонстрации работы приложения в рамках курсовой.

- (В перспективе возможен переход на PostgreSQL 17 — мощную реляционную СУБД с поддержкой JSONB и сложных аналитических запросов).

Дополнительные инструменты:

- Git — система контроля версий для отслеживания изменений в проекте.
- Visual Studio Code — основная среда разработки (с расширениями Python, Flask, HTML/CSS, Prettier и GitLens).

Обоснование выбора стека

Для курсовой работы выбран стек **Python 3.12 + Flask 3.1 + SQLite + Bootstrap 5.3 + Chart.js**.

Данный стек обеспечивает оптимальный баланс между простотой реализации, читаемостью кода и качеством конечного продукта в условиях ограниченного времени курсовой работы.

1.3 Формулировка цели и задач работы

Цель работы — разработать веб-приложение для учёта расходов и доходов семейного бюджета, обеспечивающее удобный совместный контроль финансов, прозрачность трат и наглядную аналитику для всех членов семьи.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Проанализировать существующие программные продукты и технологии разработки веб-приложений для управления семейными финансами.
2. Определить требования целевой аудитории и спроектировать функциональность приложения (диаграммы вариантов использования, User Story).
3. Разработать модель данных (ER-диаграмму, логическую и физическую схемы базы данных).
4. Создать макеты пользовательского интерфейса (wireframes).
5. Реализовать систему аутентификации и авторизации пользователей с разделением прав доступа по ролям.

6. Разработать CRUD-интерфейс для основных сущностей (транзакции, категории, бюджеты).
7. Реализовать механизмы загрузки и выгрузки файлов (чеки, отчёты) и систему агрегации данных с визуализацией статистики.
8. Провести тестирование приложения, выполнить деплой и оформить отчётную документацию.

В результате работы будет создано полноценное веб-приложение, соответствующее всем требованиям курсового проекта.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ

2.1 Анализ целевой аудитории

Основными пользователями разрабатываемой системы являются сотрудники логистических компаний. Выделяются две ключевые роли:

- **Диспетчер-логист** — сотрудник, ответственный за распределение заказов и контроль их выполнения в реальном времени.
- **Курьер** — конечный исполнитель, использующий мобильную версию приложения для навигации по маршруту.

Для каждой роли проведено профилирование по методу персонажей (Personas) для минимизации когнитивной нагрузки на интерфейс.

2.2 Описание функциональности приложения

Функциональные возможности системы определяются через взаимодействие пользователя с системой. Основной бизнес-кейс представлен в виде *User Story*: «Я, как диспетчер, хочу импортировать список адресов, чтобы система автоматически построила кратчайший маршрут для 5 курьеров».

На рисунке 2.1 представлена диаграмма вариантов использования (Use Case).

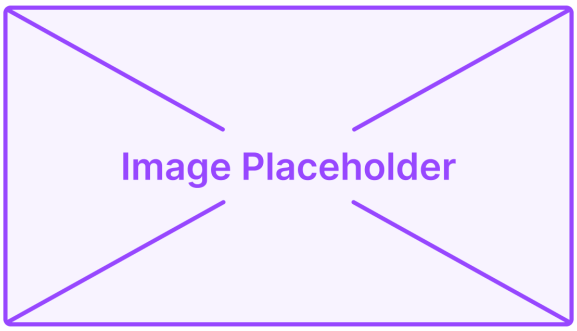

Рисунок 2.1 – Диаграмма вариантов использования системы

2.3 Проектирование модели данных

При выборе архитектуры базы данных был проведен сравнительный анализ эффективности различных решений, опирающийся на данные о произ-

водительности современных СУБД [3].

Структура таблицы заказов представлена в табл. 2.1.

Таблица 2.1 – Структура таблицы заказов (`orders`)

Поле	**Тип**	**Ограничение**	**Описание**
id	UUID	PRIMARY KEY	Уникальный идентификатор заказа
customer_id	UUID	FOREIGN KEY	Ссылка на таблицу клиентов
address	TEXT	NOT NULL	Полный адрес доставки
lat	NUMERIC	NOT NULL	Широта точки (WGS-84)
lng	NUMERIC	NOT NULL	Долгота точки (WGS-84)
weight	FLOAT	CHECK > 0	Вес груза, кг
volume	FLOAT	NOT NULL	Объем груза, $м^3$
time_from	TIME	NOT NULL	Начало окна доставки
time_to	TIME	NOT NULL	Конец окна доставки
priority	INT	DEFAULT 1	Приоритет (1–5)
status	VARCHAR	’pending’	Статус (ожидание, в пути, выполнен)
courier_id	UUID	NULL	Назначенный исполнитель
created_at	TIMESTAMP	DEFAULT NOW()	Время создания записи
updated_at	TIMESTAMP	DEFAULT NOW()	Время последнего изменения
is_fragile	BOOLEAN	FALSE	Хрупкий груз
contact_phone	VARCHAR	NOT NULL	Телефон получателя
price	MONEY	NOT NULL	Стоимость доставки
external_id	VARCHAR	UNIQUE	ID из внешней системы (1C/ERP)

Продолжение таблицы 2.1

Поле	**Тип**	**Ограничение**	**Описание**
notes	TEXT	NULL	Комментарий для курьера
attempts	INT	DEFAULT 0	Количество попыток доставки

Общая стоимость маршрута для службы доставки рассчитывается следующим образом:

$$C_{total} = \sum_{i=1}^{n} (L_i \cdot P_{fuel} + T_i \cdot W_{hour}) + S_{fixed}, \tag{1}$$

где

L_i — протяженность i-го участка маршрута, км;

P_{fuel} — стоимость топлива на 1 км пути;

T_i — время в пути на участке;

W_{hour} — почасовая ставка оплаты труда курьера;

S_{fixed} — постоянные издержки на обслуживание автопарка.

2.4 Разработка макетов страниц

Макеты страниц

ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ

3.1 Разработка базовой структуры приложения и вёрстка

В соответствии с задачей 3.1, была реализована компонентная структура фронтенд-части на базе библиотеки **React.js**. Использование хуков (*Hooks*) позволило эффективно управлять состоянием формы заказа. Пример базового компонента представлен в листинге 1.

Листинг 1 – Пример структуры компонента карточки заказа

```
// Комментарий
const OrderCard = ({ order }) => {
  return (
    <div className="order-card">
      <h3Заказ> #{order.id}</h3>
      <pАдрес>: {order.address}</p>
      <button onClick={() => addToRoute(order.id)}>
        Добавить в маршрут
      </button>
    </div>
  );
};
```

3.2 Реализация модуля авторизации

Согласно задаче 3.2, в системе реализована ролевая модель доступа. Для обеспечения безопасности используется протокол **JWT** (JSON Web Token). При входе в систему сервер генерирует токен, содержащий поле `role`, которое может принимать значения `'dispatcher'` или `'courier'`.

3.3 Интеграция картографических сервисов

Ключевым этапом разработки (задача 3.4) стала интеграция API **OpenStreetMap** через библиотеку *Leaflet*. Для решения транспортной задачи был выбран жадный алгоритм с последующей оптимизацией методом локального поиска.

Данные о маршруте передаются в формате JSON (см. листинг 2).

Как видно из листинга 2, каждая точка содержит порядковый номер (`sequence`), что позволяет курьеру следовать по оптимальному пути, рассчитанному на сервере.

Листинг 2 – Структура данных оптимизированного маршрута

```
{
  "route_id": "r-101",
  "courier_id": "u-45",
  "waypoints": [
    {"lat": 55.75, "lng": 37.61, "sequence": 1},
    {"lat": 55.80, "lng": 37.70, "sequence": 2}
  ],
  "total_distance_km": 12.4
}
```

ЗАКЛЮЧЕНИЕ

В результате проведенного исследования подтверждена гипотеза о том, что автоматизация логистических процессов на базе специализированного веб-приложения позволяет существенно повысить эффективность работы служб экспресс-доставки. Теоретический анализ показал, что классическая задача маршрутизации транспортных средств (VRP) в современных условиях требует адаптации под динамические изменения городского трафика и специфику «последней мили».

На основе полученных при разработке данных можно сформулировать следующие **выводы**:

1. Использование открытых ГИС-технологий и компонентного подхода (React/Node.js) обеспечивает необходимую архитектурную гибкость, позволяя интегрировать алгоритмы оптимизации без зависимости от проприетарных платных сервисов.

2. Проектирование модели данных с поддержкой пространственных расширений (PostGIS) является критическим условием для масштабируемости системы, так как позволяет эффективно обрабатывать геозапросы в режиме реального времени.

3. Внедрение эвристических методов построения маршрутов демонстрирует устойчивый экономический эффект, выраженный в снижении совокупного пробега автотранспорта и сокращении времени планирования рейсов диспетчером.

Рекомендации и предложения. Для практического применения разработанного решения в деятельности малых логистических компаний предлагается использовать модель облачного развертывания. Это обеспечит бесперебойный доступ к интерфейсу управления и мобильному клиенту курьера при минимальных затратах на ИТ-инфраструктуру.

Перспективы углубления темы. В дальнейшем исследовании планируется развитие функциональности системы за счет внедрения методов предиктивной аналитики для прогнозирования времени прибытия (ETA) с учетом исторических данных о заторах, а также разработки модуля динамиче-

ской корректировки маршрута при поступлении срочных заявок в процессе выполнения рейса.

При разработке пояснительной записки к курсовому проекту строго соблюдались требования [4], определяющие структуру и правила оформления научно-исследовательских работ.

СПИСОК ЛИТЕРАТУРЫ

[1] Индекс финансовой грамотности — 2025: россияне стали хуже считать проценты [Электронный ресурс], Аналитический центр НАФИ. — [Б. м. : б. и.], 2026. — URL: https://nafi.ru/polls/indeks-finansovoy-gramotnosti-2025-rossiyane-stali-khuzhe-schitat-protsenty/ (дата обращения: 01.04.2026).

[2] Банк России принял решение снизить ключевую ставку [Электронный ресурс], Центральный банк Российской Федерации. — [Б. м. : б. и.], 2026. — 03. — URL: https://www.cbr.ru/press/pr/?file=20032026_133000key.htm (дата обращения: 01.04.2026).

[3] Петров, П. П. Анализ производительности современных СУБД [Текст] / П. П. Петров, С. С. Сидоров // Вестник информационных технологий. — 2024. — № 2. — С. 15–22.

[4] ГОСТ 7.32-2017. Отчет о научно-исследовательской работе. Структура и правила оформления [Текст]. — 2017.